

Computer Vision Project Report

1. Introduction

This project report describes a two-player Rock Paper Scissors (RPS) game using OpenCV and traditional computer vision techniques. The program uses webcam input to detect the player's hand, classify the gesture, and determine the winner. Afterwards, the program enters the 'winner' mode, where the winner can choose to capture their face and perform photo editing which can then be displayed on the winner screen. The main objectives of the program are as follows:

Objective	Implementation
Build an interactive RPS game	Real-time webcam input with two players
Detect two hands simultaneously	Fixed regions of interest (ROI) for each player
Segment hand regions	Background subtraction, thresholding, and morphology, with YOLO validation
Classify gestures	Rule-based features and contour analysis, with XGBoost validation
Decide winner	RPS game logic
Track winner's face after win	Haar Cascade classifier → CamShift tracking, with CNN emotion tracking
Photo capture and editing	Brightness, contrast, saturation, grayscale, and sharpening adjustments

2. Approach and Pipeline

2.1. Pipeline Overview

- Webcam input → mirror the frame using horizontal flip → create clean processing frame and separate display frame → draw hand boxes
- Press B to save empty background → use background subtraction to detect scene changes → convert difference image to grayscale → apply Gaussian blur then binary thresholding
- Clean mask using morphological opening and closing → find hand contours → analyze contour, centroid, convex hull, and convexity defects
- Classify gestures (traditional computer vision with XGBoost validation) → start countdown after pressing S → lock gestures → decide winner
- Emotion labeling using CNN
- Switch to the 'winner' mode → detect face once using Haar cascade → CamShift tracking
- Capture face or full frame → edit photo → show winner screen

2.2. Course Concepts

OpenCV video processing⁵: `cv.VideoCapture(0)` captures webcam frames. **Image display and drawing**^{3,5}: `cv.imshow()`, `cv.rectangle()`, `cv.putText()`, and `cv.circle()` display overlays. **Image manipulation**^{1,2}: Frames are flipped, copied, cropped, resized, and edited. **Region of Interest**^{4,5}: Two fixed hand boxes are used for each player. **Background subtraction**: Detects scene changes. **Grayscale conversion**^{2,3}: Difference images are converted from BGR to grayscale. **Gaussian blur**¹: Reducing noise before thresholding. **Thresholding**²: Creates hand

masks. **Morphology**³: Opening and closing. **Contour detection**³: `cv.findContours()` extracts hand boundaries. **Contour area filtering**³: Small contours are ignored as noise. **Convex hull**³: `cv.convexHull()` creates a boundary around the hand. **Convexity defects**: `cv.convexityDefects()` counts gaps between fingers. **Moments**³: `cv.moments()` calculates the hand centroid. **Shape analysis**: Solidity, defect count, and contour area are used for gesture classification. **Object detection**⁵: Haar Cascade detects face. **Object tracking**⁵: CamShift tracks face. **Back projection**⁵: CamShift uses HSV histogram back projection for tracking. **Image enhancement**¹: Photo editor. **Filtering/convolution**^{1,2}: Sharpening uses a 3x3 kernel.

2.3. Implementation Details

2.3.1. Hand Detection and Contour Analysis

We use background subtraction for hand detection. The user presses B to save the empty background. The program then compares the current ROI with the saved background. The program creates a mask, where white represents hand/foreground and black represents the background. It is used for contour detection.

The process is current ROI → absolute difference + grayscale + Gaussian blur + binary thresholding → morphological opening then closing → the final hand mask is generated.

After creating the binary mask, the program uses `cv.findContours()` to find object boundaries. The program selects the contour with the largest area because the hand should be the largest object inside the ROI, while small contours are ignored because they usually come from noise, shadows, or minor segmentation errors. For each valid hand contour, the program draws the hand contour, centroid, convex hull, defect count, and gesture label. The centroid is calculated using image moments: $cx = \frac{m10}{m00}$, $cy = \frac{m01}{m00}$.

Convexity defect is calculated from the difference between the actual contour and the convex hull. This represents the gaps between fingers.

2.3.2. Gesture Classification

The program uses rule-based classification and traditional shape analysis to determine the gesture. It calculates the contour area, convex hull area, solidity, and convexity defect count, where $solidity = contour\ area / convex\ hull\ area$.

The classification rules are as follows:

- Rock: Up to 1 defect, > 0.80 solidity (compact shape)
- Paper: At least 3 defects (open hand shape)
- Scissors: 1–2 defects (two fingers extended)
- Unknown: Otherwise, i.e. no valid contour, contour is too small, or unclear hand shape

2.3.3. Winner Face Detection and Tracking

If there is a decisive result, the program switches to the 'winner' mode. The hand boxes, contours, masks, defect counts, and game text are removed from the display, returning to a clean camera frame with only facial capture instructions. Facial detection works by cleaning the camera frame → converting to grayscale → applying Haar Cascade classifier `detectMultiScale()` → getting the face boxes → choosing the largest face, since the winner is expected to stand close to the camera.

CamShift is used in conjunction with Haar detection because it allows us to follow the face after initial detection. The tracking window changes based on the color distribution of the tracked face region. The detected face box is used to

initialize CamShift tracking: Haar Cascade detects face once → crop detected face ROI → convert ROI to HSV → calculate hue-saturation histogram → normalize histogram. We calculate the histogram back projection to apply CamShift and update the face box for each new frame.

2.3.4. Photo Capture, Editing, and Winner Screen

If the user presses F, the program crops the tracked face box from the clean camera frame. Padding is added around the face, so the result looks better. If the user presses P, the program captures the full clean camera frame. The user can perform the following adjustments:

- Brightness and contrast: `new_pixel = old_pixel * contrast + brightness`
- Saturation: Convert to HSV and scaling the saturation channel
- Grayscale: Converts the image to grayscale, then converts it back to BGR for display.
- Sharpening: Uses a Laplacian-style sharpening filter. It increases the local contrast around edges and emphasizes details. The following 3x3 kernel is used: `[ [0, -1, 0], [-1, 5, -1], [0, -1, 0] ]`.

After the photo is saved, the program displays the winner screen, which includes the edited photo

2.3.5. Machine Learning Integration^{6,7}

To validate results and provide additional features, the program uses 3 machine learning components including YOLO, XGBoost classifier, and CNN. YOLO is used for object detection and localization, It provides bounding boxes and class labels (hand or face), which is used as input for XGBoost and CNN models. XGBoost uses numerical features extracted from hand contours, and it is used to classify gestures. Lastly, CNN uses face images to classify emotions. It is suitable for this task because facial expressions are represented by patterns like eye shapes and mouth curves that is fit for automatic feature learning.

3. Performance Evaluation

The program worked well when the environment was controlled, especially when the lighting was stable and the background did not change after calibration. Since the hand detection depends on background subtraction, the result was best when both hand boxes were empty before pressing 'B' which is the background calibration key. After calibration, the program was able to separate the hand from the background and create a binary mask for contour.

The preprocessing steps, including Gaussian blur, thresholding, and morphology, helped reduce noise in the hand mask. Contour detection, convex hull, and convexity defects were then used to analyze the hand shape. During testing, rock and paper were usually more stable because their shapes are clearer. Scissors were harder to classify sometimes because the gap between fingers can change depending on the hand angle and lighting.

YOLO and XGBoost improved the program by giving extra support to the detection and classification steps. YOLO was useful for confirming that a hand was present, while XGBoost helped support the gesture result when contour features were clear. Both still depended on good image quality, correct preprocessing, and stable lighting.

The program was able to run in real time while detecting hands, classifying gestures, deciding the winner, and switching to the winner photo mode. The countdown also helped make the result more stable because it locked the gestures instead of changing the result every frame.

For the winner photo feature, Haar Cascade worked well when the winner faced the camera clearly. CamShift helped track the face after detection, so the program did not need to detect the face again every frame. The photo editor also worked smoothly because the edits were applied to a captured image instead of live video.

However, the project still has some limitations. The hand detection can become noisy if the lighting changes or if the background moves after calibration. The fixed hand boxes also mean players must keep their hands inside the correct area. Gesture classification may fail when the contour is unclear, and CamShift may drift if the background has colors similar to the face.

4. Originality Statement

This project was created as an original group project for the Computer Vision course. The main idea was to build an interactive Rock Paper Scissors game that detects players' hand gestures using webcam input, decides the winner, and lets the winner capture and edit a photo.

The main system was built using traditional Computer Vision techniques from the course, such as real-time video capture, ROI processing, background subtraction, thresholding, morphology, contour detection, convex hull analysis, convexity defects, moments, Haar Cascade face detection, CamShift tracking, and image enhancement. These parts were combined into one complete application instead of being used as separate small examples.

The project was not copied from an existing online project. External libraries such as OpenCV, NumPy, Ultralytics YOLO, and XGBoost were used as tools to support the implementation. YOLO was included as a supporting component for hand detection and confirmation, while XGBoost was added as a supporting feature-based method for gesture classification. The main system still depended on traditional Computer Vision techniques.

5. Conclusion

This project demonstrates how traditional Computer Vision techniques can be used to create a complete real-time interactive application. The system uses webcam input to detect two players' hands, classify their gestures, decide the winner, track the winner's face, and allow photo editing after the game.

The project applies many topics from the course, including image segmentation, thresholding, morphology, contour detection, shape analysis, object detection, object tracking, and image enhancement. These techniques were combined into one practical system rather than only being tested separately.

Overall, the project achieved its main goal of creating a webcam-based Rock Paper Scissors game using Computer Vision. The system works best in a stable environment, but it still has limitations with lighting changes, fixed hand boxes, and unclear gestures. Future improvements could include adaptive background subtraction, stronger hand tracking, and further improvement of the ML-based gesture classification to support the current traditional Computer Vision approach.

6. Usage Instructions

Clone the [GitHub repository](#), install Python 3.12, install required packages: `pip install opencv-python numpy ultralytics`, run the program: `python main.py`

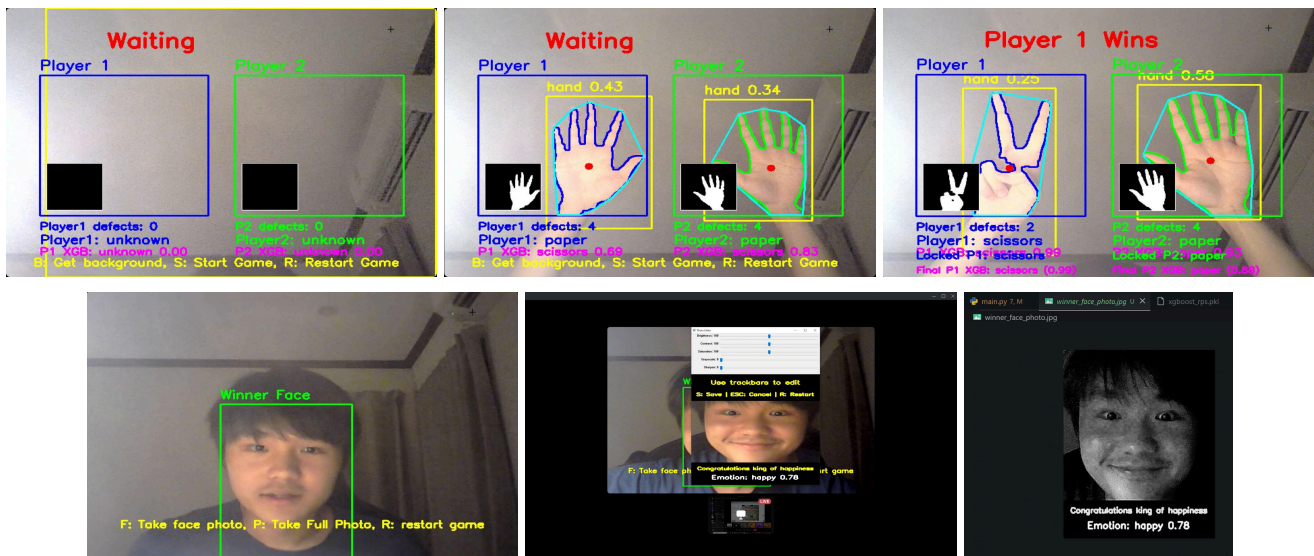
6.1. Controls

Control	Action (winner mode actions are underlined)
B	Save empty background

S	Start game, <u>save photo</u>
R	Restart game
F	<u>Take face photo</u>
P	<u>Take full camera photo</u>
Esc	Quit, <u>cancel editing</u>

6.2. Usage steps

1. Run `main.py`
2. Keep both hand boxes empty
3. Press B to save the background
4. Place both hands inside the boxes
5. Press S to start the round
6. Wait for the countdown
7. After a winner is decided, position your face in front of the camera
8. Press F for face photo or P for full frame photo
9. Edit the photo
10. Press S to save photo
11. View the winner screen
12. Press R to restart or Esc to quit



Appendix

References

- 1 - Lecture 7: Image Restoration, Noise Filters, and Pyramids; 2 - Lecture 8: Image Segmentation and Thresholding; 3 - Lecture 9: Compression, Contours, Shapes, and Morphological Operations; 4 - Lecture 10: Feature Description, Descriptors, and Keypoint Algorithms; 5 - Lecture 11: Feature Matching, Object Detection, and Tracking Algorithms; 6 - Lecture 12: ML and DL; 7 - Lecture 13: CNN

Member List

6638076921 Thienkwan Boonchai, 6738210721 Poomison Yongyingsakthaworn, 6738236021 Win Pinyopasakul, 6738237721 Wittapatch Dulayasopapan